

chunyuac_HW5

October 21, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

Question One

Reading data in and show the first few data points:

```
[1]: import pandas as pd

bonds = pd.read_csv('bonds.txt', sep='\t', index_col=0)
# bonds = bonds[bonds['CouponRate'] > 6]
n = len(bonds)

bonds.head()
```

```
[1]:      CouponRate  BidPrice
Case
1         7.000      92.94
2         9.000     101.44
3         7.000      92.66
4         4.125      94.50
5        13.125     118.94
```

There are 35 data points.

```
[2]: n
```

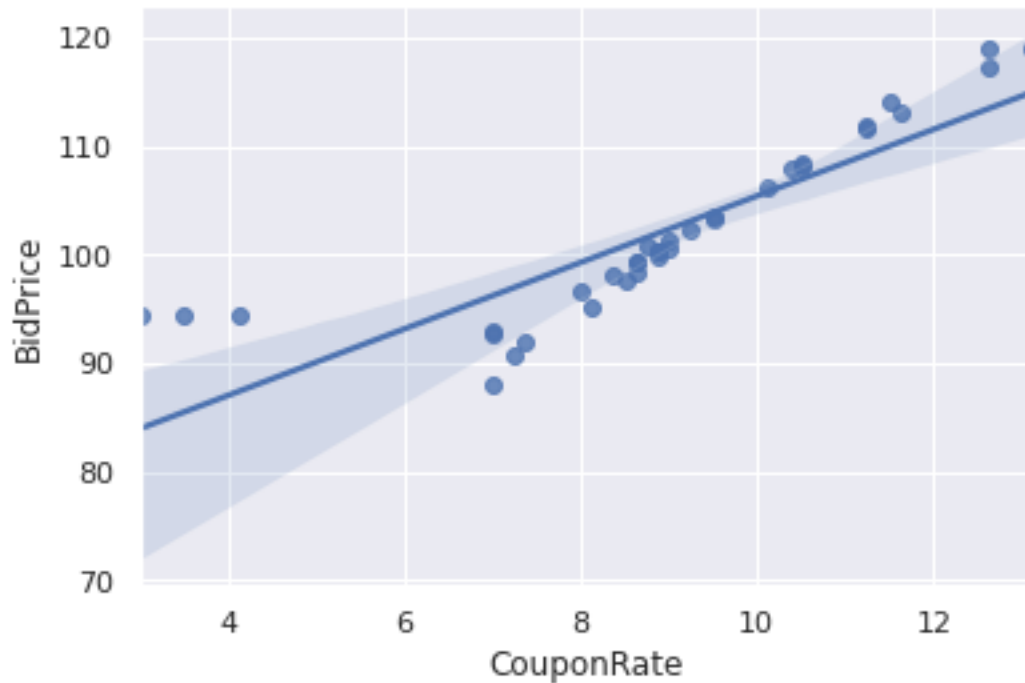
```
[2]: 35
```

1.

The below figure suggests that there might be outliers for low coupon Rates. Linear regression model does not fit the data very well.

```
[3]: import seaborn as sns
```

```
sns.set_theme(color_codes=True)
ax = sns.regplot(x='CouponRate', y='BidPrice', data=bonds)
```



2.

See below for the summary of the fitted model.

```
[4]: import statsmodels.api as sm
import matplotlib.pyplot as plt

olsBonds = sm.OLS.from_formula(formula='BidPrice ~ CouponRate', data=bonds).
    ↪fit()
olsBonds.summary()
```

```
[4]: <class 'statsmodels.iolib.summary.Summary'>
"""
```

```

                                OLS Regression Results
=====
Dep. Variable:                  BidPrice    R-squared:                   0.752
Model:                            OLS      Adj. R-squared:              0.744
Method:                 Least Squares    F-statistic:                   99.87
Date:                Thu, 15 Oct 2020    Prob (F-statistic):           1.64e-11
Time:                  09:01:31          Log-Likelihood:                -98.656
```

```

No. Observations:      35    AIC:      201.3
Df Residuals:          33    BIC:      204.4
Df Model:              1
Covariance Type:      nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      74.7866      2.827      26.458      0.000      69.036      80.537
CouponRate      3.0661      0.307       9.994      0.000       2.442       3.690
=====
Omnibus:                3.313    Durbin-Watson:           1.827
Prob(Omnibus):          0.191    Jarque-Bera (JB):        2.262
Skew:                   0.607    Prob(JB):                0.323
Kurtosis:               3.276    Cond. No.                37.3
=====

```

Notes:

```

[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
"""

```

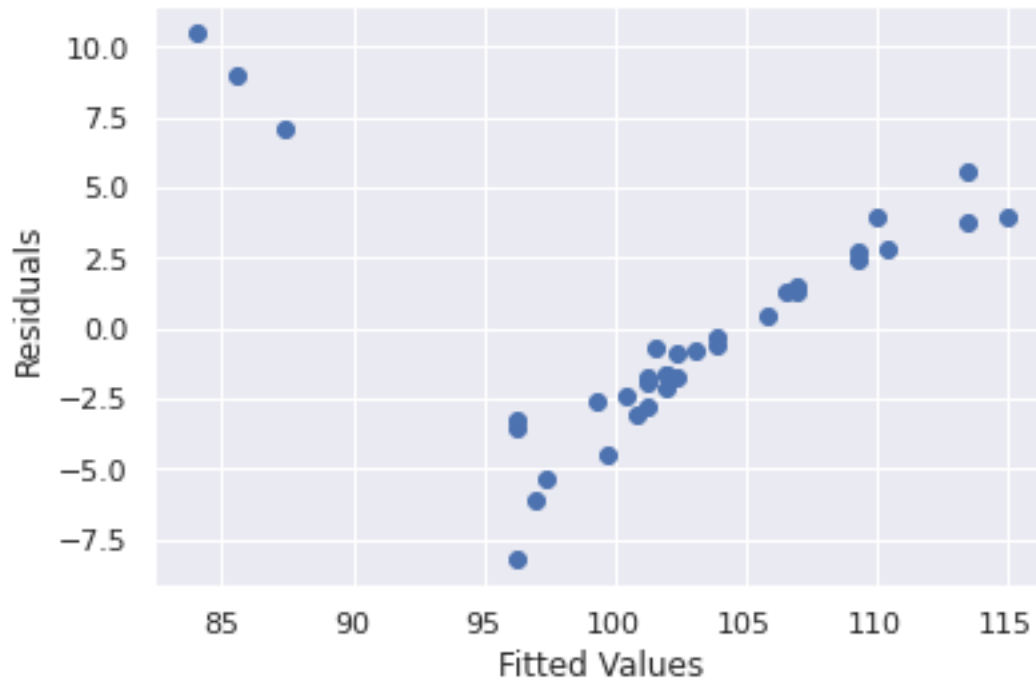
3.

We see clear pattern in the plot of residuals versus fitted values, suggesting the linear model is not adequate.

```

[5]: plt.scatter(olsBonds.fittedvalues, olsBonds.resid)
      plt.xlabel('Fitted Values')
      plt.ylabel('Residuals')
      plt.show()

```



4.

Recall that the 95% confidence interval is $\hat{\beta}_1 \pm z_{0.025} \text{SE}(\hat{\beta}_1)$, where the standard error is

$$\text{SE}(\hat{\beta}_1) \approx \frac{\hat{\sigma}}{s_x \sqrt{n-1}}$$

and

$$\hat{\sigma}^2 = \text{RSS}/(n-2).$$

Below is the numerical value. Here the normal distribution is used. If we use t -distribution instead, the confidence interval will exactly match the one reported in the summary above. As $n > 30$, the differences are tiny.

```
[6]: import numpy as np
from scipy.stats import norm, t

se = np.sqrt(olsBonds.ssr/(n-2))/(np.std(bonds['CouponRate'], ddof=1)*np.
    ↳sqrt(n-1))
# z = t.ppf(1 - 0.025, n-2)
z = norm.ppf(1 - 0.025)

(olsBonds.params['CouponRate'] - (z*se), olsBonds.params['CouponRate'] + (z*se))
```

```
[6]: (2.464778252846198, 3.6674260585957033)
```

The derivation of the confidence interval is based on the assumption that the linear model is adequate, which is not the case for the data set here. Thus the confidence interval is not meaningful.

Question Two

1.

Read in the data and drop the id and bond_id columns which clearly have no predictive power.

```
[7]: data = pd.read_csv('bonddata.txt', sep=',')
data.head()
```

```
[7]:      id  bond_id  trade_price  weight  current_coupon  time_to_maturity  \
0    10         1    126.268   7.616851         5.950        25.940349
1    38         2    101.009   0.018373         3.550         0.326392
2   336         3    108.144   3.537380         4.200         2.340601
3   781         4    116.953   0.402523         4.875         7.367653
4  1172        14    133.210   0.210685        10.750         4.159301
```

```
      is_callable  reporting_delay  trade_size  trade_type  ...  \
0              0          12.540   150000.0         3  ...
1              0           9.327   450000.0         3  ...
2              0           0.102   160000.0         4  ...
3              0          52.093   10000.0         4  ...
4              0          13.722   25000.0         4  ...
```

```
      received_time_diff_last9  trade_price_last9  trade_size_last9  \
0              2862729         128.596        120000.0
1              532644         101.086        500000.0
2              96293         107.917        400000.0
3              92559         115.924         10000.0
4             6400208         127.500         25000.0
```

```
      trade_type_last9  curve_based_price_last9  received_time_diff_last10  \
0              2         127.561201        3621492
1              2         101.103180        538510
2              2         107.874965         96293
3              2         116.160420         97177
4              2         131.066998        6400208
```

```
      trade_price_last10  trade_size_last10  trade_type_last10  \
0          129.089        200000.0         2
1          101.070         25000.0         2
2          107.917        400000.0         4
3          116.458         2000.0         3
4          127.500         25000.0         4
```

```

    curve_based_price_last10
0          129.435587
1          101.102243
2          107.874965
3          116.342802
4          131.066998

```

[5 rows x 61 columns]

```
[8]: data.drop(['bond_id', 'id'], axis='columns', inplace=True)
```

From Kaggle’s description of the data, `is_callable` and any column with “type” in the column name is categorical. These columns need to be converted into dummy variables for regression. Calling `astype('category')` on those columns is not necessary. Once the conversion to dummy variables is done, `astype('category')` will not make any difference.

After the conversion, there are 82 predictors in the data.

```
[9]: import pandas as pd

catCols = ['is_callable'] + [colName for colName in data.columns if 'type' in
    ↪ colName]

# for colName in catCols:
#     data[colName] = data[colName].astype('category')

data = pd.get_dummies(data=data, columns=catCols)
data.shape
```

[9]: (1620, 82)

2.

Below is the model fitted using all predictors. From the plot of residuals versus fitted values there seem to be a few outliers in the data.

```
[10]: import statsmodels.api as sm

X = sm.add_constant(data.drop('trade_price', axis='columns'))
fullmod = sm.OLS(endog=data['trade_price'], exog=X).fit()

plt.scatter(fullmod.fittedvalues, fullmod.resid)
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
```

```
plt.show()
```

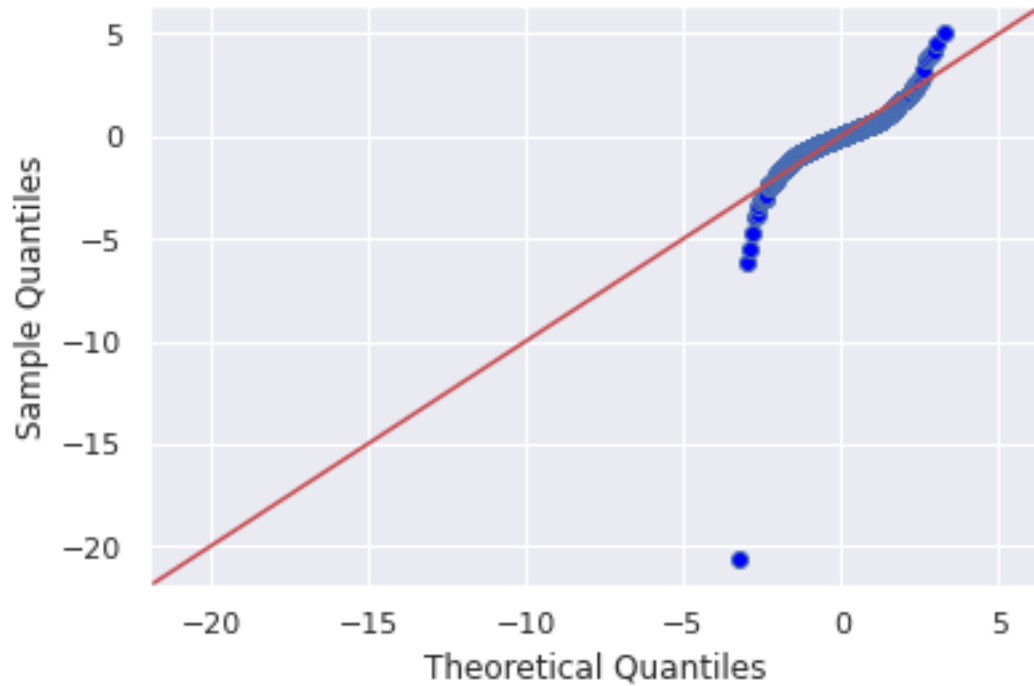


3.

From the below normal probability plot, it is clear the residual distribution has heavier tail than normal, and the left tail seems to be heavier. Also, there is a clear outlier in the data.

```
[11]: from statsmodels.graphics.gofplots import qqplot

qqplot(fullmod.resid, dist=norm, line='45', fit=True)
plt.show()
```



4., 5.

Here is the stepwise variable selection procedure using AIC/PRESS.

```
[12]: X.drop(['const'], axis='columns', inplace=True)
```

```
[13]: from statsmodels.stats import outliers_influence

def AIC(model):
    return model.aic

def PRESS(model):
    levs = outliers_influence.OLSInfluence(model).hat_matrix_diag
    return ((model.resid/(1 - levs))**2).sum()

def modelSelection(endog, exog, selectionMethod=AIC):
    """
    Assuming no constant column in the input exog
    """
    chosen = []
    remaining = list(exog.columns.values)
    best = np.inf
```



```

    for i in range(len(X.columns)):
        idx_quantities = [(idx, selectionMethod(sm.OLS(endog, exog=sm.
→add_constant(exog[[feature] + chosen])).fit())) for idx, feature in
→enumerate(remaining)]
        idx, q = min(idx_quantities, key=lambda idx_q: idx_q[1])
        if q >= best:
            break
        best = q
        chosen.append(remaining[idx])
        remaining = remaining[:idx] + remaining[idx+1:]

    return chosen

chosenAIC = modelSelection(endog=data['trade_price'], exog=X,
→selectionMethod=AIC)
chosenPRESS = modelSelection(endog=data['trade_price'], exog=X,
→selectionMethod=PRESS)

chosenAIC, chosenPRESS

```

```

[13]: (['trade_price_last1',
        'curve_based_price',
        'trade_type_2',
        'trade_price_last7',
        'trade_type_3',
        'trade_price_last10',
        'trade_price_last4',
        'curve_based_price_last5',
        'trade_type_last1_3',
        'trade_type_last4_3',
        'trade_type_last4_2',
        'trade_type_last1_2',
        'trade_price_last9',
        'trade_type_last9_2',
        'curve_based_price_last1',
        'curve_based_price_last4',
        'curve_based_price_last3',
        'received_time_diff_last10',
        'trade_type_last5_3',
        'trade_price_last3',
        'is_callable_1',
        'reporting_delay'],
        ['trade_price_last1',
        'curve_based_price',
        'trade_type_2',
        'trade_price_last7',

```

```

'trade_type_4',
'trade_price_last10',
'trade_type_last1_3',
'curve_based_price_last5',
'trade_price_last4',
'trade_type_last4_3',
'trade_type_last1_2',
'trade_type_last4_2',
'trade_size_last7',
'is_callable_1',
'trade_size',
'received_time_diff_last10',
'trade_price_last9',
'trade_type_last9_2',
'reporting_delay',
'trade_type_last5_3',
'received_time_diff_last8',
'trade_size_last4',
'trade_size_last5',
'trade_type_last1_4',
'trade_type_3'])

```

The selection process is computationally intense. We copy and paste the results below. Using AIC, the procedure selected 22 features, while using PRESS, it selected 25. There are 18 common features selected by both methods.

```

[14]: chosenAIC = ['trade_price_last1', 'curve_based_price', 'trade_type_2',
↳ 'trade_price_last7', 'trade_type_3', 'trade_price_last10',
↳ 'trade_price_last4', 'curve_based_price_last5', 'trade_type_last1_3',
↳ 'trade_type_last4_3', 'trade_type_last4_2', 'trade_type_last1_2',
↳ 'trade_price_last9', 'trade_type_last9_2', 'curve_based_price_last1',
↳ 'curve_based_price_last4', 'curve_based_price_last3',
↳ 'received_time_diff_last10', 'trade_type_last5_3', 'trade_price_last3',
↳ 'is_callable_1', 'reporting_delay']
chosenPRESS = ['trade_price_last1', 'curve_based_price', 'trade_type_2',
↳ 'trade_price_last7', 'trade_type_4', 'trade_price_last10',
↳ 'trade_type_last1_3', 'curve_based_price_last5', 'trade_price_last4',
↳ 'trade_type_last4_3', 'trade_type_last1_2', 'trade_type_last4_2',
↳ 'trade_size_last7', 'is_callable_1', 'trade_size',
↳ 'received_time_diff_last10', 'trade_price_last9', 'trade_type_last9_2',
↳ 'reporting_delay', 'trade_type_last5_3', 'received_time_diff_last8',
↳ 'trade_size_last4', 'trade_size_last5', 'trade_type_last1_4', 'trade_type_3']

len(chosenAIC), len(chosenPRESS), len(set(chosenAIC) & set(chosenPRESS))

```

```

[14]: (22, 25, 18)

```

Below are the summary of both models. They both have some features with coefficients very close

to zero. If we take away those features, the resulting models will be almost identical.

```
[15]: model = sm.OLS(endog=data['trade_price'], exog=sm.add_constant(X[chosenAIC])).
      ↪fit()
      model.summary()
```

```
[15]: <class 'statsmodels.iolib.summary.Summary'>
      """
```

```

                                OLS Regression Results
=====
Dep. Variable:                trade_price    R-squared:                0.980
Model:                        OLS           Adj. R-squared:        0.980
Method:                       Least Squares  F-statistic:             3527.
Date:                         Thu, 15 Oct 2020 Prob (F-statistic):       0.00
Time:                         09:06:58      Log-Likelihood:         -3092.3
No. Observations:             1620          AIC:                  6231.
Df Residuals:                 1597          BIC:                  6355.
Df Model:                     22
Covariance Type:              nonrobust
=====
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
const                1.2270    0.452      2.713    0.007
0.340    2.114
trade_price_last1     0.4340    0.028    15.456    0.000
0.379    0.489
curve_based_price     0.3929    0.038    10.296    0.000
0.318    0.468
trade_type_2         -0.8080    0.100    -8.112    0.000
-1.003   -0.613
trade_price_last7     0.1129    0.023     4.843    0.000
0.067    0.159
trade_type_3          0.8038    0.100     8.027    0.000
0.607    1.000
trade_price_last10    0.0376    0.026     1.422    0.155
-0.014    0.090
trade_price_last4     0.1033    0.030     3.392    0.001
0.044    0.163
curve_based_price_last5 -0.2031    0.050    -4.037    0.000
-0.302   -0.104
trade_type_last1_3    -0.2174    0.100    -2.169    0.030
-0.414   -0.021
trade_type_last4_3    -0.2073    0.103    -2.017    0.044
-0.409   -0.006

```

trade_type_last4_2	0.2007	0.104	1.929	0.054
-0.003	0.405			
trade_type_last1_2	0.2293	0.106	2.157	0.031
0.021	0.438			
trade_price_last9	0.0763	0.029	2.641	0.008
0.020	0.133			
trade_type_last9_2	0.1874	0.100	1.883	0.060
-0.008	0.383			
curve_based_price_last1	-0.0607	0.051	-1.180	0.238
-0.162	0.040			
curve_based_price_last4	0.2077	0.074	2.795	0.005
0.062	0.354			
curve_based_price_last3	-0.1591	0.066	-2.419	0.016
-0.288	-0.030			
received_time_diff_last10	-3.692e-08	2.06e-08	-1.791	0.074
-7.74e-08	3.52e-09			
trade_type_last5_3	-0.1440	0.088	-1.628	0.104
-0.317	0.029			
trade_price_last3	0.0469	0.031	1.534	0.125
-0.013	0.107			
is_callable_1	0.1791	0.120	1.498	0.134
-0.055	0.413			
reporting_delay	-1.75e-06	1.22e-06	-1.429	0.153
-4.15e-06	6.53e-07			

Omnibus:	1930.311	Durbin-Watson:	1.978
Prob(Omnibus):	0.000	Jarque-Bera (JB):	907009.858
Skew:	-5.538	Prob(JB):	0.00
Kurtosis:	118.389	Cond. No.	3.10e+07

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.1e+07. This might indicate that there are strong multicollinearity or other numerical problems.

"""

```
[16]: model = sm.OLS(endog=data['trade_price'], exog=sm.add_constant(X[chosenPRESS])).
      ↪ fit()
      model.summary()
```

```
[16]: <class 'statsmodels.iolib.summary.Summary'>
      """
```

```

                                OLS Regression Results
=====
Dep. Variable:                  trade_price    R-squared:                  0.980
```

```

Model:                      OLS      Adj. R-squared:      0.979
Method:                    Least Squares      F-statistic:      3361.
Date:                      Thu, 15 Oct 2020      Prob (F-statistic):      0.00
Time:                      09:06:58      Log-Likelihood:      -3094.8
No. Observations:          1620      AIC:      6238.
Df Residuals:              1596      BIC:      6367.
Df Model:                  23
Covariance Type:          nonrobust

```

```

=====
=====

```

	coef	std err	t	P> t
[0.025 0.975]				

const	0.7032	0.274	2.564	0.010
0.165 1.241				
trade_price_last1	0.4291	0.026	16.782	0.000
0.379 0.479				
curve_based_price	0.3540	0.028	12.554	0.000
0.299 0.409				
trade_type_2	-0.5763	0.114	-5.042	0.000
-0.800 -0.352				
trade_price_last7	0.1113	0.023	4.748	0.000
0.065 0.157				
trade_type_4	0.2491	0.105	2.383	0.017
0.044 0.454				
trade_price_last10	0.0486	0.026	1.853	0.064
-0.003 0.100				
trade_type_last1_3	0.0175	0.107	0.163	0.871
-0.193 0.228				
curve_based_price_last5	-0.1643	0.029	-5.654	0.000
-0.221 -0.107				
trade_price_last4	0.1400	0.025	5.677	0.000
0.092 0.188				
trade_type_last4_3	-0.2342	0.103	-2.272	0.023
-0.436 -0.032				
trade_type_last1_2	0.4564	0.120	3.810	0.000
0.221 0.691				
trade_type_last4_2	0.2174	0.106	2.060	0.040
0.010 0.424				
trade_size_last7	-8.056e-08	5.48e-08	-1.469	0.142
-1.88e-07 2.7e-08				
is_callable_1	0.1951	0.120	1.631	0.103
-0.040 0.430				
trade_size	7.488e-08	5.71e-08	1.312	0.190
-3.71e-08 1.87e-07				
received_time_diff_last10	-1.072e-07	6.32e-08	-1.697	0.090

-2.31e-07	1.67e-08				
trade_price_last9		0.0709	0.029	2.463	0.014
0.014	0.127				
trade_type_last9_2		0.1980	0.100	1.980	0.048
0.002	0.394				
reporting_delay		-1.664e-06	1.23e-06	-1.355	0.176
-4.07e-06	7.44e-07				
trade_type_last5_3		-0.1471	0.089	-1.655	0.098
-0.321	0.027				
received_time_diff_last8		8.794e-08	7.06e-08	1.245	0.213
-5.06e-08	2.27e-07				
trade_size_last4		-7.847e-08	5.67e-08	-1.385	0.166
-1.9e-07	3.26e-08				
trade_size_last5		6.841e-08	6.04e-08	1.133	0.258
-5.01e-08	1.87e-07				
trade_type_last1_4		0.2293	0.103	2.220	0.027
0.027	0.432				
trade_type_3		1.0304	0.109	9.426	0.000
0.816	1.245				
=====					
Omnibus:		1943.345	Durbin-Watson:		1.976
Prob(Omnibus):		0.000	Jarque-Bera (JB):		923080.057
Skew:		-5.607	Prob(JB):		0.00
Kurtosis:		119.402	Cond. No.		1.03e+21
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 2.04e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

"""